MÔN NHẬN DẠNG MẪU (PATTERN RECOGNITION)

Nhận dạng chữ số viết tay MNIST

Bốn bộ phân lớp tự cài đặt: Gaussian Naïve Bayes, hồi quy softmax, MLP và AdaBoost-SAMME

Đồ án giữa kỳ

So sánh trên cùng một giao thức không rò rỉ dữ liệu, có đối chiếu với scikit-learn, demo ONNX chạy trong trình duyệt.

Thành viên	MSSV	Đóng góp
Trần Trinh Nhân	24280088	50%
Lâm Nhật Tiến	24280029	50%

Mã nguồn: `https://github.com/TranTrinhNhan1/WriteRHand`

Ngày biên dịch: 8/11/2026

Tóm tắt

Chúng tôi tự cài đặt bốn bộ phân lớp cho MNIST: Gaussian Naïve Bayes, hồi quy softmax huấn luyện bằng Gradient Descent, MLP với lan truyền ngược viết tay và AdaBoost-SAMME trên cây quyết định có trọng số. Phần tính toán lõi dùng NumPy; scikit-learn chỉ để đối chiếu, PyTorch chỉ làm thư viện tensor cho MLP. Cả bốn mô hình dùng chung một giao thức không rò rỉ dữ liệu: 50 000 ảnh train, 10 000 ảnh validation tách phân tầng từ tập huấn luyện chính thức, và 10 000 ảnh test chỉ dùng để báo cáo. Siêu tham số được chọn trên validation bằng các lượt quét từng tham số, toàn bộ nhật ký được giữ lại.

Trên tập test, độ chính xác lần lượt là 81.32% (Naïve Bayes), 92.52% (softmax), 98.24% (MLP) và 97.60% (AdaBoost). MLP đứng đầu, và hơn AdaBoost có ý nghĩa thống kê trên cùng 10 000 ảnh (McNemar chính xác, $p = 4 \times 10^{-6}$). AdaBoost mất 817.9 s để huấn luyện, hơn bảy mươi lần so với MLP (11.0 s), và chậm nhất khi dự đoán từng ảnh (18.705 ms). Kết quả của bản tự cài đặt lệch scikit-learn tối đa 0.66 điểm phần trăm trên cùng cấu hình. Mô hình được xuất sang ONNX, đối chiếu số học với bản Python và chạy trực tiếp trong trình duyệt. Chúng tôi cũng nêu những giới hạn của kết quả: một seed cho mô hình chính, quét tuần tự chưa xét tương tác giữa các siêu tham số, và thời gian đo trên một máy.

Từ khóa: MNIST, Naïve Bayes, hồi quy softmax, MLP, AdaBoost, SAMME, ONNX.

Mục lục

1 Giới thiệu **3**

2 Dữ liệu và tiền xử lý **3**

3 Cơ sở lý thuyết và cài đặt **4**
3.1 Gaussian Naïve Bayes 4
3.2 Hồi quy logistic đa lớp (softmax) huấn luyện bằng Gradient Descent 4
3.3 Mạng nơ-ron nhiều lớp (MLP) và lan truyền ngược 5
3.4 AdaBoost đa lớp (SAMME) với cây quyết định có trọng số 5
3.5 Độ phức tạp tính toán 6

4 Thiết kế thí nghiệm **6**
4.1 Quy trình đánh giá thống nhất 6
4.2 Chiến lược tìm siêu tham số 6
4.3 Phần cứng, đo thời gian và theo dõi thí nghiệm 7

5 Kiểm chứng tính đúng đắn **7**

6 Kết quả **7**
6.1 Siêu tham số được chọn 8
6.2 Độ chính xác và các độ đo 9
6.3 Thời gian 10
6.4 So sánh công bằng với scikit-learn 10
6.5 Đường cong học 11

7 Thảo luận **11**
7.1 Độ chính xác, thời gian và khả năng giải thích 11
7.2 Những chữ số bị nhầm 12
7.3 Giới hạn của kết quả 13

8 Mở rộng: tăng cường dữ liệu và bản demo web **13**

8.1 Tăng cường dữ liệu cho MLP 13
8.2 Bản demo web và quy trình triển khai 14

9 Kết luận **14**

10 Đóng góp của nhóm và công bố sử dụng AI **14**
10.1 Phân công và đóng góp 14
10.2 Công bố sử dụng công cụ AI 15

A Các lượt quét siêu tham số còn lại **17**

B Tái lập kết quả **18**

1 Giới thiệu

Nhận dạng chữ số viết tay trên MNIST [7] là bài toán phân lớp 10 lớp kinh điển của nhận dạng mẫu: mỗi ảnh xám 28×28 chứa một chữ số thuộc $\{0, 1, \ldots, 9\}$ đã được căn giữa. Đồ án này *tự cài đặt* bốn bộ phân lớp thuộc bốn họ phương pháp khác nhau – mô hình sinh (Gaussian Naïve Bayes), mô hình tuyến tính phân biệt (hồi quy softmax huấn luyện bằng Gradient Descent), mạng nơ-ron nhiều lớp (MLP, lan truyền ngược) và ensemble (AdaBoost-SAMME) – rồi so sánh chúng trên cùng một giao thức đánh giá về độ chính xác, thời gian huấn luyện, thời gian dự đoán và khả năng giải thích.

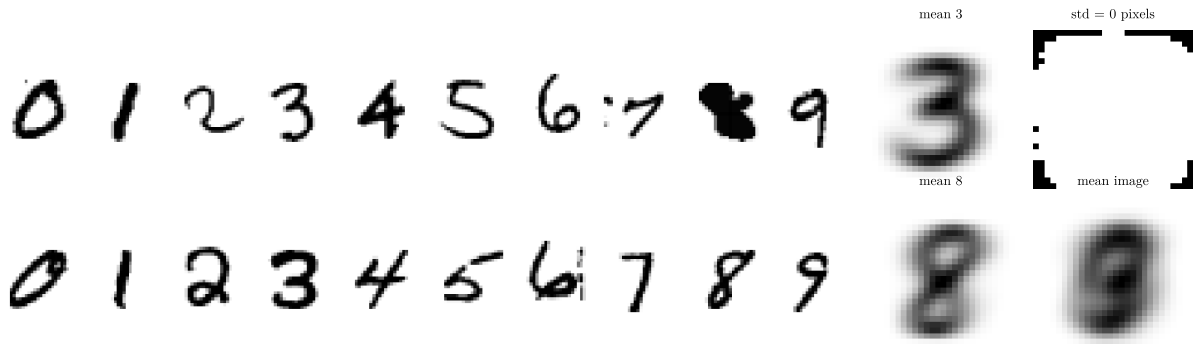
Quy ước "from scratch". Phần tính toán lõi của mỗi thuật toán (ước lượng tham số, hàm mất mát, gradient, cập nhật, dựng cây, cập nhật trọng số boosting) do nhóm viết bằng NumPy. scikit-learn chỉ được dùng để *đối chiếu* (kiểm thử và so sánh baseline); PyTorch chỉ cung cấp tensor và sinh số ngẫu nhiên cho MLP, *không dùng autograd để huấn luyện*.

Đóng góp chính. (i) Bốn bộ phân lớp với giao diện thống nhất, có kiểm thử đối chiếu scikit-learn và kiểm tra gradient bằng sai phân hữu hạn; (ii) một giao thức thực nghiệm nhất quán, không rò rỉ dữ liệu, điều khiển bằng dòng lệnh và ghi nhận bằng Weights & Biases; (iii) các thí nghiệm siêu tham số với đầy đủ nhật ký, so sánh công bằng với scikit-learn; (iv) bản demo web chạy hoàn toàn trên trình duyệt (ONNX Runtime Web) và quy trình xuất/kiểm chứng mô hình; (v) một nghiên cứu mở rộng có đối chứng về tăng cường dữ liệu. Toàn bộ mã nguồn, kết quả và notebook nằm trong kho GitHub của đồ án; mô hình đã huấn luyện được đưa lên Hugging Face Hub (Mục 8.2).

Bố cục. Mục 2 mô tả dữ liệu; Mục 3 trình bày lý thuyết và cài đặt; Mục 4 giao thức thực nghiệm; Mục 5 kiểm chứng; Mục 6 kết quả; Mục 7 thảo luận; các mục cuối là mở rộng, kết luận, đóng góp nhóm và công bố việc sử dụng AI.

2 Dữ liệu và tiền xử lý

MNIST gồm 60 000 ảnh huấn luyện và 10 000 ảnh kiểm thử chính thức, mỗi ảnh 28×28 điểm ảnh 8 bit (0 = nền đen, 255 = nét trắng), chữ số đã được chuẩn hóa kích thước về hộp 20×20 và căn giữa theo trọng tâm [7]. Dữ liệu được tải bằng `torchvision.datasets.MNIST`.



Hình 1. Khảo sát dữ liệu (tập huấn luyện chính thức): hai hàng mẫu ngẫu nhiên cho mỗi chữ số, ảnh trung bình của lớp 3 và 8, bản đồ pixel không đổi, ảnh trung bình toàn tập.

Phân bố lớp. Các lớp gần cân bằng: lớp 1 chiếm nhiều nhất (khoảng 11,2%) và lớp 5 ít nhất (khoảng 9,0%), nên accuracy là chỉ số hợp lý, nhưng vẫn báo cáo macro precision/recall/F1 để lớp ít mẫu không bị che khuất.

Pixel không mang thông tin. Khoảng 81% giá trị điểm ảnh là nền (0). Trên 50 000 ảnh huấn luyện có 71 vị trí *luôn* là nền (phương sai bằng 0), tập trung ở viền ảnh. Chúng làm Naïve Bayes

có phương sai 0 (cần smoothing, Mục 3) và không bao giờ được cây quyết định chọn để tách.

Chia tập và chuẩn hóa. Như trong Mục 4: chia phân tầng 50 000/10 000 (seed 42, mã băm `52cc8206ff96f56d`), chuẩn hóa vô hướng với $\bar{x} = 0{,}13063$ và $s = 0{,}30808$ (tính trên phần train). Dùng một cặp $(\bar{x}, s)$ chung cho cả ảnh (thay vì chuẩn hóa từng pixel) giữ nguyên hình dạng của dữ liệu, tránh chia cho phương sai gần 0 của các pixel ở viền và cho phép bản demo trên trình duyệt áp dụng đúng cùng phép biến đổi. Ảnh được làm phẳng thành vector 784 chiều.

3 Cơ sở lý thuyết và cài đặt

Ký hiệu: $\boldsymbol{x} \in \mathbb{R}^D$ ($D = 784$) là ảnh đã chuẩn hóa, $y \in \{0, \dots, K-1\}$ ($K = 10$) là nhãn, tập huấn luyện $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$. Mọi thuật toán dưới đây có *phần tính toán lõi do nhóm tự viết* bằng NumPy (và PyTorch chỉ như thư viện tensor cho MLP, không dùng autograd); scikit-learn chỉ xuất hiện trong các bài kiểm thử và phần đối chiếu (Mục 5). Giao diện chung (`fit`, `predict`, `predict_proba`, `save`/`load`) nằm trong `src/models/base.py`.

3.1 Gaussian Naïve Bayes

Mô hình sinh với giả thiết độc lập có điều kiện giữa các pixel khi biết lớp, mỗi pixel theo phân phối chuẩn [2, 4]:

$$p(\boldsymbol{x} \mid y = k) = \prod_{j=1}^{D} \mathcal{N}\!\left(x_j; \mu_{kj}, \sigma_{kj}^2\right), \qquad \hat{y} = \arg\max_k \Big[\log \pi_k - \tfrac{1}{2}\sum_{j=1}^{D}\Big(\log(2\pi\sigma_{kj}^2) + \tfrac{(x_j-\mu_{kj})^2}{\sigma_{kj}^2}\Big)\Big]. \tag{1}$$

Ước lượng hợp lý cực đại (MLE) cho ra $\pi_k = N_k/N$, μ_{kj} là trung bình mẫu và σ_{kj}^2 là phương sai mẫu (chia cho N_k) của pixel j trong lớp k. Các pixel luôn bằng 0 có phương sai 0 làm $\log \sigma^2 \to -\infty$, nên cộng thêm $\varepsilon = \texttt{var_smoothing} \cdot \max_j \mathrm{Var}(x_j)$ vào mọi phương sai (cùng quy ước với scikit-learn). Việc tính trong không gian log tránh underflow khi nhân 784 thừa số; xác suất hậu nghiệm được chuẩn hóa bằng log-sum-exp. Huấn luyện chỉ cần một lượt qua dữ liệu, $O(ND)$; dự đoán $O(NKD)$. Siêu tham số duy nhất là `var_smoothing`.

3.2 Hồi quy logistic đa lớp (softmax) huấn luyện bằng Gradient Descent

Với $\boldsymbol{z} = W^\top \boldsymbol{x} + \boldsymbol{b}$, $p_k = \exp(z_k)/\sum_m \exp(z_m)$, hàm mất mát trên một mini-batch $\mathcal{B}$ gồm cross-entropy và phạt L2 (không phạt bias):

$$L(W, \boldsymbol{b}) = -\frac{1}{|\mathcal{B}|}\sum_{i \in \mathcal{B}} \log p_{y_i}(\boldsymbol{x}_i) + \frac{\lambda}{2}\|W\|_F^2. \tag{2}$$

Đạo hàm giải tích: $\partial L/\partial z_k = p_k - \mathbb{1}[y = k]$ nên với $X \in \mathbb{R}^{B \times D}$, P là ma trận xác suất và Y là ma trận one-hot,

$$\nabla_W L = \tfrac{1}{B} X^\top (P - Y) + \lambda W, \qquad \nabla_{\boldsymbol{b}} L = \tfrac{1}{B}\,\mathbf{1}^\top (P - Y). \tag{3}$$

Softmax và cross-entropy được cài ở dạng ổn định số học (trừ $\max_k z_k$ trước khi lấy mũ, dùng $\log\sum\exp$ nên không bao giờ tính $\log 0$). Thuật toán cập nhật $\theta \leftarrow \theta - \eta \nabla_\theta L$ theo mini-batch với dữ liệu được xáo trộn mỗi epoch; hàm mục tiêu là lồi nên không cần khởi tạo ngẫu nhiên (khởi tạo bằng 0). Siêu tham số: tốc độ học η, kích thước batch, λ, số epoch. Lịch sử loss/accuracy theo epoch được ghi lại để theo dõi hội tụ.

3.3 Mạng nơ-ron nhiều lớp (MLP) và lan truyền ngược

Kiến trúc $\boldsymbol{a}_0 = \boldsymbol{x}$, $\boldsymbol{z}_l = \boldsymbol{a}_{l-1}W_l + \boldsymbol{b}_l$, $\boldsymbol{a}_l = \mathrm{ReLU}(\boldsymbol{z}_l) \odot \boldsymbol{m}_l$ cho các lớp ẩn, lớp cuối cho logits và softmax; mất mát là cross-entropy trung bình. Trọng số khởi tạo He [6]: $W_l \sim \mathcal{N}(0, 2/\mathrm{fan_in})$, bias bằng 0. Dropout dạng *inverted* [10]: $m_{l,j} \in \{0,\, 1/(1-p)\}$, chỉ bật khi huấn luyện. Lan truyền ngược [9] viết tay (không dùng autograd):

$$\boldsymbol{\delta}_L = \tfrac{1}{B}\big(\mathrm{softmax}(\boldsymbol{z}_L) - Y\big), \tag{4}$$

$$\nabla_{W_l} = \boldsymbol{a}_{l-1}^{\top}\boldsymbol{\delta}_l, \quad \nabla_{\boldsymbol{b}_l} = \textstyle\sum_{\text{hàng}} \boldsymbol{\delta}_l, \qquad \boldsymbol{\delta}_{l-1} = (\boldsymbol{\delta}_l W_l^{\top}) \odot \mathbb{1}[\boldsymbol{a}_{l-1} > 0] \odot \boldsymbol{m}_{l-1}. \tag{5}$$

Cập nhật bằng SGD có momentum [11], viết tường minh: $\boldsymbol{v} \leftarrow \mu\boldsymbol{v} + \boldsymbol{g}$, $\theta \leftarrow \theta - \eta\boldsymbol{v}$ (trùng với `torch.optim.SGD`, được kiểm thử). *Early stopping* [8]: 8% tập huấn luyện được tách nội bộ làm tập dừng, theo dõi cross-entropy (tắt dropout), kiên nhẫn 5 epoch, ngưỡng cải thiện 10^{-4}, khôi phục trọng số tốt nhất; do đó tập validation của đồ án không tham gia vào quá trình huấn luyện. Mỗi lần `fit` khởi tạo mạng và bộ sinh số ngẫu nhiên (`torch.Generator`) riêng theo `seed`, nên kết quả tái lập được.

3.4 AdaBoost đa lớp (SAMME) với cây quyết định có trọng số

Chúng tôi chọn boosting vì nó là thuật toán ensemble thật sự (các bộ học yếu được huấn luyện tuần tự và có trọng số), không phải lấy trung bình các mô hình khác. SAMME [12] mở rộng AdaBoost [5] cho K lớp: khởi tạo $w_i = 1/N$; ở vòng t, huấn luyện cây h_t với trọng số mẫu w, tính sai số có trọng số $\mathrm{err}_t = \sum_i w_i \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]$ (với w đã chuẩn hóa) và

$$\alpha_t = \eta\Big(\ln\frac{1-\mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1)\Big), \tag{6}$$

$$w_i \leftarrow w_i \exp\big(\alpha_t \mathbb{1}[h_t(\boldsymbol{x}_i) \neq y_i]\big) \text{ rồi chuẩn hóa}, \tag{7}$$

$$H(\boldsymbol{x}) = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(\boldsymbol{x}) = k]\,. \tag{8}$$

Số hạng $\ln(K-1)$ khiến bộ học chỉ cần tốt hơn đoán ngẫu nhiên ($\mathrm{err} < 1 - 1/K$) thay vì $< 1/2$; vòng lặp dừng khi điều kiện này bị vi phạm. η là hệ số co (`learning_rate`).

Bộ học yếu: cây CART độ sâu $\le d_{\max}$ giảm thiểu tạp chất Gini có trọng số $G(S) = W_S - \sum_k W_{S,k}^2/W_S$ [3], với $W_S = \sum_{i\in S} w_i$, $W_{S,k} = \sum_{i\in S,\, y_i=k} w_i$. Để huấn luyện nhanh trên $50\,000\times784$, mỗi đặc trưng được lượng tử hóa *một lần* thành n_{bins} khoảng đều nhau; ở mỗi tầng của cây, trọng số theo (nút, đặc trưng, bin, lớp) được cộng dồn bằng histogram, tổng tích lũy cho phép đánh giá mọi ngưỡng của mọi đặc trưng trong $O(ND)$ mỗi tầng. Ngưỡng thực được lưu cho dự đoán nên suy luận không cần lượng tử hóa. Tuỳ chọn `max_features` chọn ngẫu nhiên một tập con đặc trưng cho mỗi cây.

Thuật toán: AdaBoost-SAMME (`src/models/adaboost.py`)

1. Lượng tử hóa X thành bin (một lần); $w_i \leftarrow 1/N$.
2. Với $t = 1, \ldots, T$: dựng cây h_t từ histogram có trọng số (chọn cặp (đặc trưng, ngưỡng) tối thiểu $G(L) + G(R)$ ở mỗi nút, dừng khi không giảm được tạp chất hoặc đạt $d_{\max}$).
3. $\mathrm{err}_t \leftarrow \sum_i w_i[h_t(\boldsymbol{x}_i) \neq y_i]$; nếu $\mathrm{err}_t \ge 1 - 1/K$ thì dừng.
4. $\alpha_t \leftarrow \eta\,(\ln\frac{1-\mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1))$.
5. $w_i \leftarrow w_i\, e^{\alpha_t[h_t(\boldsymbol{x}_i)\neq y_i]}$, chuẩn hóa $\sum_i w_i = 1$.
6. Dự đoán: $\hat{y} = \arg\max_k \sum_t \alpha_t[h_t(\boldsymbol{x}) = k]$; "xác suất" là tỉ lệ phiếu bầu có trọng số (không được hiệu chỉnh).

3.5 Độ phức tạp tính toán

Mô hình	Huấn luyện	Dự đoán 1 mẫu
Gaussian NB	$O(ND)$	$O(KD)$
Softmax regression	$O(E\,N\,K\,D)$	$O(KD)$
MLP (H nơ-ron ẩn, 1 lớp)	$O(E\,N\,D\,H)$	$O(DH + HK)$
AdaBoost (T cây, độ sâu d)	$O(T\,d\,N\,D)$	$O(T\,d)$

Bảng 1. Độ phức tạp theo số mẫu N, số chiều D, số lớp K, số epoch E. Suy luận của AdaBoost chỉ so sánh Td ngưỡng nên rất rẻ.

4 Thiết kế thí nghiệm

4.1 Quy trình đánh giá thống nhất

Một giao thức duy nhất được dùng cho cả bốn mô hình (mã trong `src/experiments/`, điều khiển bằng dòng lệnh `python -m src`):

1. **Chia dữ liệu**: 60 000 ảnh huấn luyện chính thức được chia *phân tầng* (giữ tỉ lệ các lớp) và có seed cố định (42) thành 50 000 ảnh *train* và 10 000 ảnh *validation*; 10 000 ảnh *test* chính thức được giữ riêng. Mã băm của phép chia được lưu cùng mọi kết quả để chứng minh tính nhất quán.
2. **Chuẩn hóa**: chia pixel cho 255 rồi chuẩn hóa $x \leftarrow (x - \bar{x})/(s + 10^{-8})$ với $\bar{x}, s$ (vô hướng) *chỉ tính trên phần train*; áp cùng $\bar{x}, s$ cho validation, test và cho bản demo trên trình duyệt. Trong K-fold, thống kê được tính lại trong từng fold nên không rò rỉ dữ liệu giữa các fold (có kiểm thử).
3. **Chọn siêu tham số** chỉ dựa trên validation (Mục 4.2). Hàm tìm kiếm thậm chí không nhận mảng test (kiểm thử tự động xác nhận).
4. **Đánh giá cuối**: huấn luyện cấu hình đã chọn trên 50 000 ảnh train, báo cáo accuracy, precision/recall/F1 macro và ma trận nhầm lẫn trên train, validation và test. Test chỉ để báo cáo, không dùng để chọn gì.

Các chỉ số được tính từ ma trận nhầm lẫn C (hàng = nhãn thật): $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_i C_{ik} - \mathrm{TP}_k$, $\mathrm{FN}_k = \sum_j C_{kj} - \mathrm{TP}_k$; precision, recall, F_1 từng lớp rồi lấy trung bình không trọng số (macro), mẫu số bằng 0 cho giá trị 0. Phần cài đặt này được đối chiếu với scikit-learn trong kiểm thử.

4.2 Chiến lược tìm siêu tham số

Notebook ban đầu định nghĩa lưới MLP gồm 960 tổ hợp × 4 fold (3 840 lần huấn luyện), quá tốn kém và chưa từng hoàn tất. Thay vào đó chúng tôi dùng *quét tuần tự từng siêu tham số* (coordinate descent một vòng): xuất phát từ cấu hình mặc định, lần lượt quét từng siêu tham số trong khi giữ các tham số khác ở giá trị tốt nhất hiện có, chọn giá trị cho validation accuracy cao nhất (hòa thì chọn giá trị đứng trước). Chi phí là $\sum_h |V_h|$ lần huấn luyện thay vì $\prod_h |V_h|$, và mỗi lần quét đúng là đồ thị "đổi một siêu tham số, giữ nguyên các yếu tố khác" mà đề bài yêu cầu. Số vòng boosting và số epoch của softmax được chọn từ *đường cong theo vòng/epoch* của một lần huấn luyện dài (cùng quỹ đạo nên tương đương huấn luyện lại). Mọi cấu hình đã chạy đều được ghi vào `results/tuning/*.json` cùng thời gian; không có kết quả nào được nhập tay. Hạn chế: coordinate descent có thể bỏ sót tương tác giữa các siêu tham số và mỗi cấu hình chỉ chạy một seed.

4.3 Phần cứng, đo thời gian và theo dõi thí nghiệm

Mọi thời gian được đo bằng `time.perf_counter` trên *CPU* (13th Gen Intel Core i7-13620H, 16 luồng logic, PyTorch 8 luồng, Python 3.12.14), không dùng GPU. Thời gian huấn luyện là thời gian chạy thực của `fit` trên 50 000 ảnh; thời gian dự đoán là trung vị của 5 lần dự đoán trọn tập 10 000 ảnh và độ trễ trung vị của 200 lần dự đoán từng ảnh riêng lẻ. Các lần chạy được ghi bằng Weights & Biases [1] (tên mô hình, siêu tham số, seed, mã commit Git, mã băm phép chia, loss/accuracy theo epoch, ma trận nhầm lẫn, thời gian, artifact mô hình); khi không có khoá API, chế độ *offline* ghi cục bộ và có thể đồng bộ sau bằng `wandb sync`; chế độ *disabled* vẫn cho kết quả JSON đầy đủ.

5 Kiểm chứng tính đúng đắn

Gồm 72 kiểm thử tự động (`pytest`), chạy trong vài chục giây trên CPU và trong CI (GitHub Actions) mà không cần tải MNIST. Dữ liệu `digits` 8×8 đi kèm scikit-learn thay MNIST trong kiểm thử.

Gaussian NB. Trung bình, phương sai, prior, nhãn dự đoán, xác suất và log-xác suất khớp scikit-learn `GaussianNB` (sai số $< 10^{-6}$) với ba giá trị `var_smoothing`; xác suất chuẩn hóa về 1 và hữu hạn ngay cả khi đầu vào cực lớn; pixel phương sai 0 không gây lỗi; kiểm tra đầu vào (NaN, sai kích thước, `var_smoothing` < 0).
Softmax regression. Gradient giải tích khớp sai phân hữu hạn trung tâm ($\epsilon = 10^{-6}$, sai số $< 10^{-7}$) có và không có L2; softmax bất biến theo phép dịch và ổn định với logits ± 1000; loss giảm đơn điệu trên dữ liệu thật; khi cả hai hội tụ, nghiệm và xác suất trùng với `LogisticRegression` (L-BFGS) của scikit-learn trên cùng hàm mục tiêu (giá trị mục tiêu chênh $< 10^{-5}$).
MLP. Trên mạng nhỏ độ chính xác kép: gradient lan truyền ngược khớp sai phân hữu hạn (không dropout và có dropout với mặt nạ cố định), khớp autograd của PyTorch (dùng *chỉ trong kiểm thử*), đạo hàm ReLU bằng 0 ở nơ-ron không hoạt động, bước SGD momentum trùng `torch.optim.SGD`, độ lệch chuẩn khởi tạo He đúng $\sqrt{2/\text{fan_in}}$, mặt nạ dropout là inverted và chỉ có khi huấn luyện, cùng seed cho trọng số trùng bit, early stopping không dùng tập validation ngoài, dự đoán khớp một bản cài đặt forward độc lập bằng NumPy, lưu/nạp cho kết quả y hệt.
AdaBoost. Cây một nút và nhiều tầng cho phân hoạch giống `DecisionTreeClassifier` (Gini, `sample_weight`) trên dữ liệu lưới nguyên; trọng số mẫu quyết định nhãn lá; công thức $\alpha_t = \ln \frac{1-\text{err}_t}{\text{err}_t} + \ln(K-1)$ được kiểm tra đúng; độ chính xác tăng khi thêm cây; trên `digits` độ chính xác chênh lệch `AdaBoostClassifier` (SAMME) không quá 6 điểm phần trăm với cùng số cây và độ sâu (hai bản cài đặt khác nhau về lượng tử hóa ngưỡng và tie-breaking nên không thể trùng tuyệt đối).
Tiền xử lý, chỉ số, pipeline. Phép chia phân tầng là một phân hoạch, giữ tỉ lệ lớp, tái lập theo seed; thống kê chuẩn hóa chỉ từ phần train; K-fold không rò rỉ; chỉ số khớp scikit-learn trên dữ liệu ngẫu nhiên và khớp ví dụ tính tay; pipeline huấn luyện/đánh giá/xuất mô hình tái lập kết quả; ONNX khớp mô hình Python (Mục 8.2).

6 Kết quả

Mọi số liệu trong mục này được đọc từ các tệp kết quả do chương trình ghi ra (`results/final`, `results/tuning`, `results/baselines`); không có con số nào nhập tay. Cấu hình của từng mô hình là cấu hình tốt nhất trên validation (bảng 2); mỗi mô hình được huấn luyện lại một lần trên 50 000 ảnh train rồi mới chạm vào tập test.

6.1 Siêu tham số được chọn

Mô hình	Siêu tham số được chọn	Số cấu hình đã chạy
Gaussian NB	`var_smoothing=0.1`	12
Softmax regression	`learning_rate=0.03, l2=0.0, epochs=67, batch_size=128`	16
MLP	`hidden_sizes=(512, 256), learning_rate=0.05, momentum=0.9, p_drop=0.0, batch_size=128, epochs=40`	22
AdaBoost-SAMME	`n_estimators=393, max_depth=10, learning_rate=0.1, n_bins=32, max_features=0.5`	14

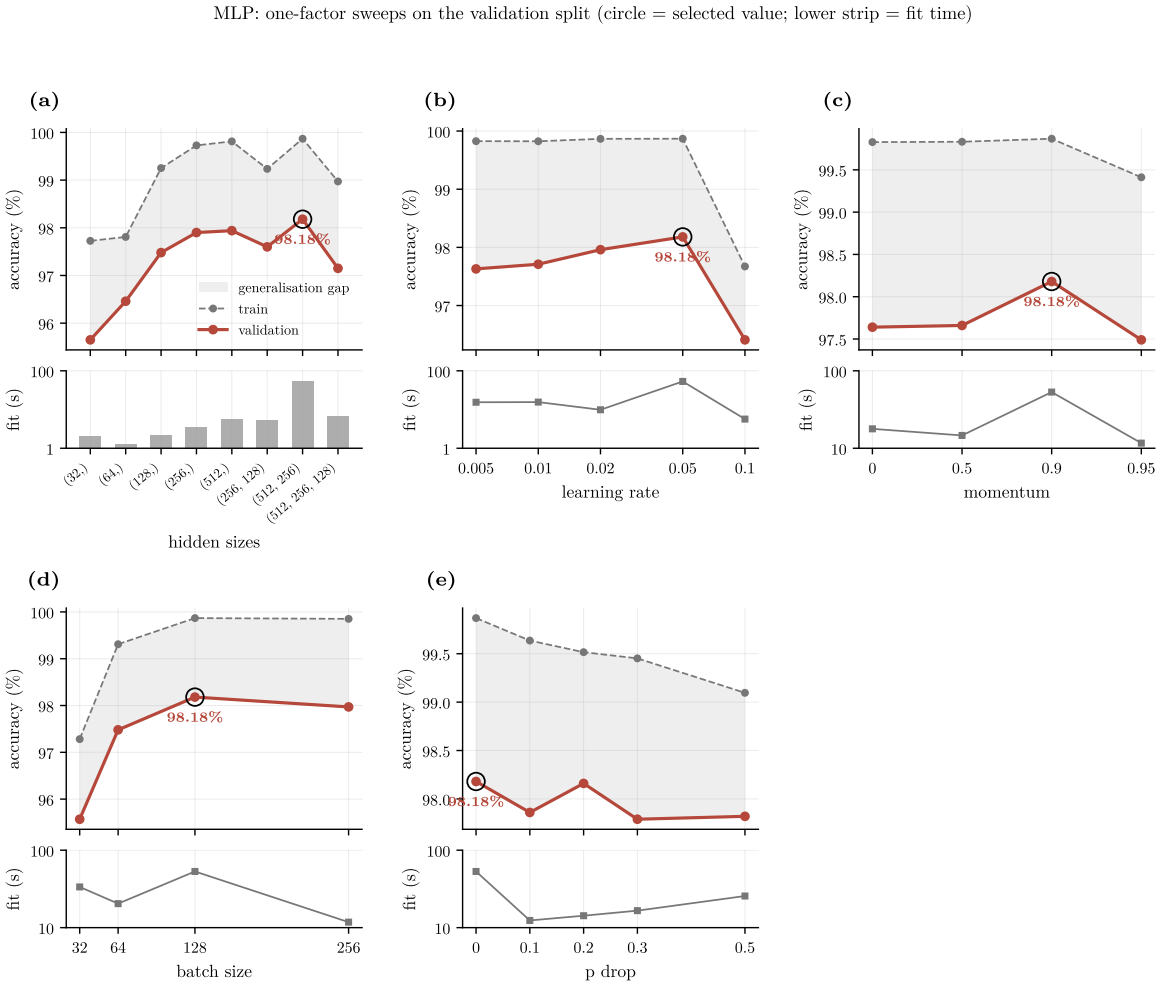
Bảng 2. Cấu hình chọn bằng validation. `epochs` của softmax và `n_estimators` của AdaBoost lấy từ đường cong validation của một lần chạy dài (tối đa 100 epoch và 400 vòng).

Bảng 3 tóm tắt từng lượt quét: khoảng dao động của validation cho thấy tham số nào thật sự quan trọng. Độ sâu cây quyết định (AdaBoost) và kích thước/hình dạng mạng (MLP) là hai tham số kéo độ chính xác nhiều nhất; với Naïve Bayes, `var_smoothing` mặc định 10^{-9} cho kết quả rất tệ (xem Mục 7).

Mô hình	Siêu tham số	Số giá trị	Val thấp nhất	Val cao nhất	Giá trị tốt nhất
Gaussian NB	`var_smoothing`	12	56.05	79.67	`0.1`
Softmax regression	`learning_rate`	6	88.03	91.72	`0.03`
Softmax regression	`batch_size`	6	90.23	91.72	`128`
Softmax regression	`l2`	5	91.20	91.76	`0.0`
MLP	`hidden_sizes`	8	95.65	98.18	`(512, 256)`
MLP	`learning_rate`	5	96.41	98.18	`0.05`
MLP	`momentum`	4	97.49	98.18	`0.9`
MLP	`batch_size`	4	95.57	98.18	`128`
MLP	`p_drop`	5	97.79	98.18	`0.0`
AdaBoost-SAMME	`max_depth`	5	77.23	95.98	`10`
AdaBoost-SAMME	`max_features`	3	95.98	96.29	`0.5`
AdaBoost-SAMME	`learning_rate`	4	96.29	97.02	`0.1`
AdaBoost-SAMME	`n_bins`	4	96.73	97.02	`32`

Bảng 3. Từng lượt quét siêu tham số trên tập validation (độ chính xác, %). Các lượt quét chạy tuần tự nên mỗi bảng ứng với phần còn lại giữ ở giá trị tốt nhất đã tìm được trước đó.

Hình 2 cho thấy các lượt quét của MLP, kèm khoảng cách giữa train và validation. Hình của ba mô hình còn lại nằm ở Mục A. Các thanh thời gian ở dải dưới chỉ mang tính tham khảo: một số lượt quét có thể chạy cùng lúc với công việc khác trên máy nên thời gian không so sánh được tuyệt đối với bảng thời gian cuối.



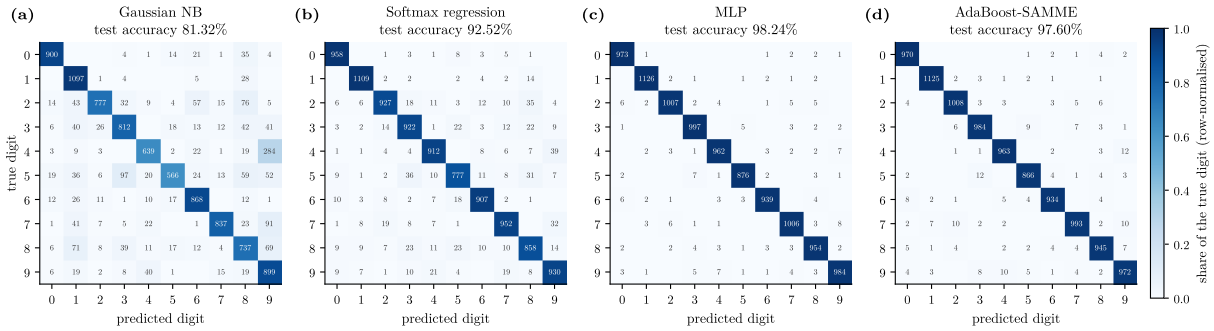
Hình 2. MLP: quét từng siêu tham số trên validation. Vòng tròn đánh dấu giá trị được chọn.

6.2 Độ chính xác và các độ đo

Mô hình	Train	Val	Test	P_{macro}	R_{macro}	$F1_{macro}$
Gaussian NB	80.13	79.67	**81.32**	82.98	80.95	81.16
Softmax regression	93.62	92.15	**92.52**	92.45	92.41	92.42
MLP	99.87	98.18	**98.24**	98.23	98.23	98.23
AdaBoost-SAMME	100.00	97.59	**97.60**	97.58	97.58	97.58

Bảng 4. Độ chính xác (%) trên train, validation, test và precision/recall/F1 macro trên test. Một seed (42) cho mỗi mô hình.

Độ chính xác test của bốn mô hình là 81.32, 92.52, 98.24 và 97.60% (bảng 4). Với $n = 10\,000$ ảnh, khoảng tin cậy nhị thức 95% cho độ chính xác quanh 98% rộng khoảng ± 0.3 điểm phần trăm, nên chênh lệch giữa MLP và AdaBoost (98.24% so với 97.60%) không nhỏ hơn nhiễu lấy mẫu một cách hiển nhiên. Vì cả hai được đánh giá trên cùng ảnh nên chúng tôi dùng kiểm định McNemar chính xác: có 127 ảnh chỉ MLP đoán đúng và 63 ảnh chỉ AdaBoost đoán đúng, $p = 4 \times 10^{-6}$. Kết luận này chỉ nói về một seed và một phép chia dữ liệu.

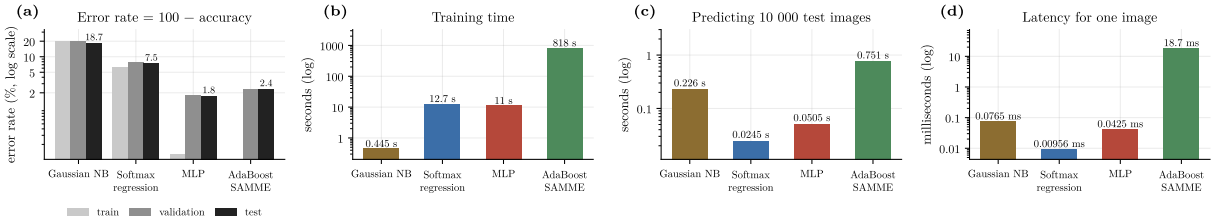


Hình 3. Ma trận nhầm lẫn trên tập test (hàng = chữ số thật, chuẩn hóa theo hàng; số trong ô là số ảnh).

6.3 Thời gian

Mô hình	Huấn luyện (s)	Dự đoán 10 000 mẫu (s)	1 mẫu (ms)
Gaussian NB	0.4	0.226	0.076
Softmax regression	12.7	0.024	0.010
MLP	11.0	0.051	0.042
AdaBoost-SAMME	817.9	0.751	18.705

Bảng 5. Thời gian trên CPU (cấu hình máy ở Mục 4.3). Huấn luyện trên 50 000 ảnh; dự đoán 10 000 ảnh test; độ trễ là trung vị của 200 lần dự đoán một ảnh.



Hình 4. Lỗi và thời gian của bốn mô hình, thang log. Vạch sáng/trung/đậm trong (a) là train/validation/test; sai số train của AdaBoost bằng 0 nên cột train không hiện trên thang log.

6.4 So sánh công bằng với scikit-learn

Với mỗi mô hình, scikit-learn được dùng *cùng siêu tham số* đã chọn ở trên, huấn luyện trên cùng 50 000 ảnh và đánh giá trên cùng tập test (không chọn lại siêu tham số cho scikit-learn). Đây là so sánh về tính đúng đắn của cài đặt hơn là cuộc đua về thuật toán.

Mô hình	Test (ours)	Test (sklearn)	$\lvert\Delta\rvert$	Fit ours (s)	Fit sklearn (s)
Gaussian NB	81.32	81.32	0.00	0.4	0.2
Softmax regression	92.52	91.86	0.66	12.7	154.6
MLP	98.24	98.02	0.22	11.0	57.8
AdaBoost-SAMME	97.60	97.50	0.10	817.9	1892.8

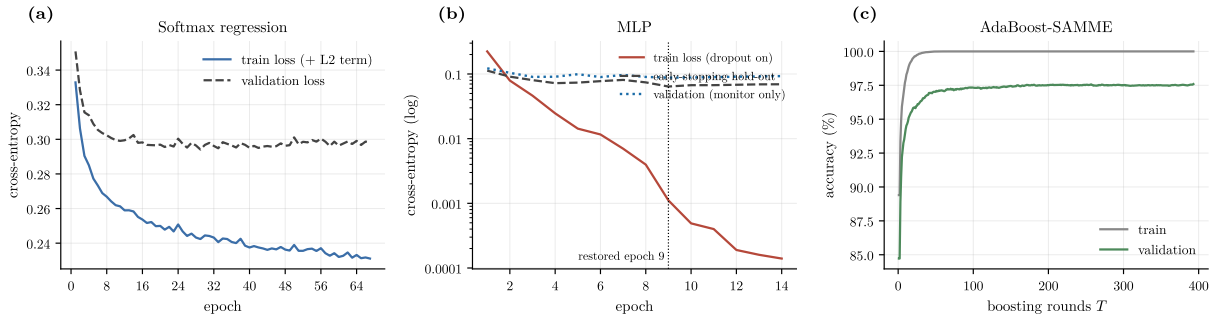
Bảng 6. Bản tự cài đặt so với scikit-learn trên tập test (độ chính xác, %) và thời gian huấn luyện.

- **Naïve Bayes** trùng khớp hoàn toàn (81.32% cả hai), đúng như kiểm thử số học ở Mục 5.

- **Softmax**: bản của chúng tôi cao hơn 0.66 điểm phần trăm. Đây không phải lợi thế thuật toán: scikit-learn chạy L-BFGS đến giới hạn 1000 vòng (cảnh báo chưa hội tụ), không dừng sớm và không chính quy hóa ($\ell_2 = 0$ như cấu hình được chọn), còn chúng tôi dừng ở epoch được chọn trên validation.
- **MLP**: 98.24% so với 98.02%. Cùng kiến trúc, momentum, batch và quy tắc dừng sớm, nhưng khởi tạo, thứ tự lấy mẫu và cách tách tập dừng sớm khác nhau nên chênh 0.22 điểm phần trăm nằm trong dao động giữa các seed (xem bảng 7).
- **AdaBoost**: 97.60% so với 97.50% (chênh 0.10). Hai bản không cùng thuật toán chia: của chúng tôi dùng histogram `n_bins`=32 và chọn ngẫu nhiên một nửa số đặc trưng cho mỗi cây, scikit-learn tìm ngưỡng chính xác trên mọi đặc trưng. Thời gian huấn luyện (817.9 s so với 1892.8 s) vì vậy không phải so sánh sạch, và lượt chạy scikit-learn có trùng thời gian với một thí nghiệm khác trên cùng máy.

Thời gian của các bản scikit-learn khác nhau chủ yếu do thư viện đại số tuyến tính và cách cài đặt (ví dụ `lbfgs` lặp 1000 vòng trên toàn bộ dữ liệu), không nên đọc thành nhận xét về thuật toán.

6.5 Đường cong học



Hình 5. (a) softmax: loss train có cộng thành phần ℓ_2 (bằng 0 ở cấu hình được chọn) và loss validation theo epoch. (b) MLP: loss train (có dropout nếu bật), loss trên tập giữ lại để dừng sớm và loss validation chỉ để theo dõi; epoch tốt nhất được khôi phục. (c) AdaBoost: độ chính xác theo số vòng boosting.

7 Thảo luận

7.1 Độ chính xác, thời gian và khả năng giải thích

Bốn mô hình nằm trên một đường đánh đổi khá rõ giữa độ chính xác, chi phí và độ dễ đọc (bảng 4 và 5).

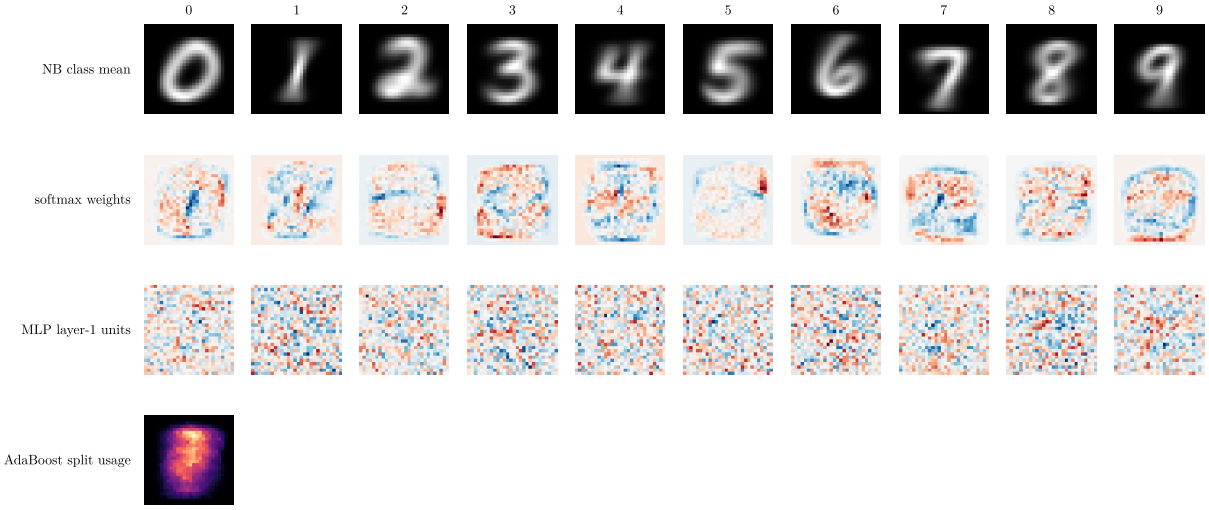
Naïve Bayes (81.32%) rẻ nhất: huấn luyện trong 0.4 s, vì chỉ cần trung bình và phương sai từng pixel theo lớp. Nhưng giả thiết các pixel độc lập có điều kiện theo lớp sai với ảnh chữ số, nơi các pixel lân cận tương quan mạnh. Hệ quả thấy rõ ở bước chọn siêu tham số: với `var_smoothing` mặc định 10^{-9} validation chỉ đạt khoảng 56%, và tăng lên 0.1 mới đạt 79.67%. Giải thích hợp lý (chúng tôi chưa kiểm chứng riêng): nhiều pixel ở rìa ảnh gần như luôn bằng 0 nên phương sai ước lượng rất nhỏ, và một pixel hiếm khi sáng lên cũng đủ kéo log-likelihood của lớp đó xuống rất thấp; làm trơn phương sai bù lại điều này. Đổi lại, mô hình dễ giải thích nhất: ảnh trung bình của từng lớp (hình 6, hàng đầu) cho biết nó "nhìn thấy" gì.

Softmax (92.52%) là mô hình tuyến tính: mỗi lớp là một mẫu (template) w_k và lớp nào có điểm $w_k^\top x + b_k$ cao nhất thì thắng. Nó nhanh (12.7 s huấn luyện, 0.010 ms mỗi ảnh) nhưng bị giới hạn bởi biên quyết định tuyến tính. Trọng số (hình 6, hàng hai) đọc được ở mức mờ: có vệt dương/âm theo nét của chữ số, song nhiễu hơn ảnh trung bình của Naïve Bayes.

MLP (98.24%) là mô hình tốt nhất và vẫn rẻ: 11.0 s huấn luyện vì dừng sớm ở epoch 14 (khôi

phục epoch 9). Đổi lại, các đơn vị lớp ẩn (hình 6, hàng ba) gần như không đọc được bằng mắt. Chênh lệch giữa train (99.87%) và test (98.24%) khoảng 1.63 điểm phần trăm; dropout không giúp thêm ở lượt quét (khoảng `p_drop`=0.2 chỉ sát giá trị 0, trong phạm vi nhiễu).

AdaBoost (97.60%) đạt độ chính xác cao nhưng đắt nhất mọi mặt: 817.9 s huấn luyện, 0.751 s cho 10 000 ảnh test, 18.705 ms cho một ảnh vì phải duyệt 393 cây, và tệp xuất ra nặng hơn 15 MiB. Độ chính xác train đạt 100.00% từ rất sớm (hình 5c) trong khi validation vẫn tăng chậm theo số vòng; đây là hành vi quen thuộc của boosting với cây sâu: sai số huấn luyện về 0 không có nghĩa là hết khả năng tổng quát hóa. Mô hình có một dạng giải thích riêng: tần suất các pixel được dùng để chia nhánh (hình 6, hàng cuối) tập trung ở vùng trung tâm, nơi có nét chữ. Điểm số của nó là tỉ lệ phiếu, *chưa hiệu chỉnh* thành xác suất.



Hình 6. Cái mà mỗi mô hình "học": ảnh trung bình theo lớp của Naïve Bayes, trọng số softmax (đỏ dương, xanh âm), một số đơn vị ẩn đầu tiên của MLP và bản đồ tần suất chia nhánh của AdaBoost.

7.2 Những chữ số bị nhầm

Cặp nhầm nhiều nhất của từng mô hình (nhãn thật→nhãn dự đoán, số ảnh) trong tập test:

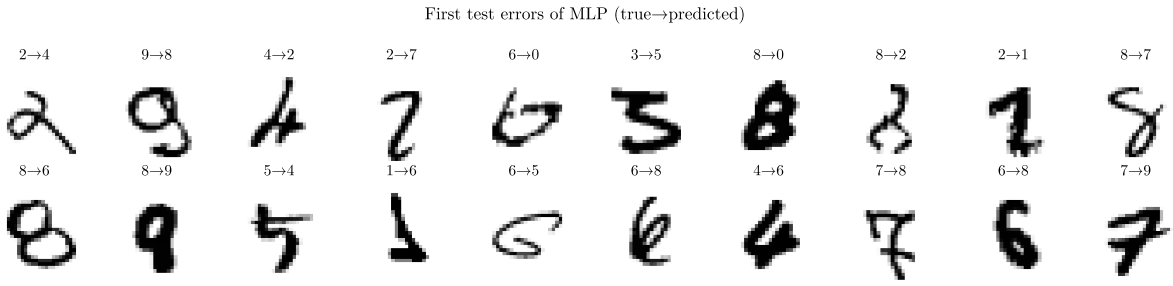
Gaussian NB. 4→9 (284), 5→3 (97), 7→9 (91).

Softmax regression. 4→9 (39), 5→3 (36), 2→8 (35).

MLP. 7→9 (8), 9→4 (7), 5→3 (7).

AdaBoost-SAMME. 5→3 (12), 4→9 (12), 7→9 (10).

Cả bốn mô hình đều nhầm $4 \to 9$ hoặc $7 \to 9$ và $5 \to 3$: những cặp có nét chung (đầu chữ 4 và chữ 9 khép hay hở, nét ngang của 7). Với Naïve Bayes riêng cặp $4 \to 9$ chiếm tới 284 trong 1868 lỗi. Với MLP, chỉ 176 ảnh sai, và một số ảnh sai là những chữ khó đọc ngay cả với người (hình 7); nên một phần lỗi còn lại có thể đến từ chính nhãn và nét chữ mơ hồ chứ không chỉ từ mô hình.



Hình 7. Hai mươi ảnh test đầu tiên MLP dự đoán sai (nhãn thật→dự đoán).

7.3 Giới hạn của kết quả

- **Một seed.** Mỗi mô hình chính chỉ được huấn luyện một lần (seed 42) và chỉ có một phép chia train/validation. Riêng MLP chúng tôi có ba seed trong thí nghiệm tăng cường (bảng 7): độ lệch chuẩn của test accuracy giữa các seed là cỡ vài phần mười điểm phần trăm, tức cùng bậc với chênh lệch giữa MLP và scikit-learn.
- **Quét tuần tự.** Coordinate descent một vòng không xét tương tác giữa các siêu tham số (ví dụ learning rate tối ưu phụ thuộc kích thước batch và momentum) và có thể bỏ lỡ tổ hợp tốt hơn. Thứ tự quét cũng ảnh hưởng đến kết quả.
- **Chọn trên validation, báo cáo trên test.** Test chỉ được dùng một lần cho mỗi cấu hình cuối cùng. Với Naïve Bayes, test cao hơn validation (81.32% so với 79.67%), điều thường thấy ở MNIST vì tập test chính thức dễ hơn một chút; chúng tôi không dùng điều này để chọn gì.
- **Một máy, CPU, có nhiễu.** Thời gian đo trên một máy; không dùng GPU. Một số lượt chạy (các baseline scikit-learn, tuning) có thể trùng thời gian với công việc khác nên chỉ dùng để so bậc độ lớn.
- **Phạm vi.** Kết quả chỉ nói về MNIST: ảnh 28×28, căn giữa, một chữ số. Bản demo web cắt, thu nhỏ và căn giữa nét vẽ theo cách của MNIST nhưng chữ viết trên màn hình vẫn khác phân phối huấn luyện, nên có thể sai ở những ca MLP không sai trên tập test.

8 Mở rộng: tăng cường dữ liệu và bản demo web

8.1 Tăng cường dữ liệu cho MLP

Câu hỏi: tăng cường dữ liệu có giúp MLP không? Chúng tôi giữ nguyên cấu hình đã chọn và chỉ đổi tập huấn luyện: gốc (50 000 ảnh) so với gốc cộng hai bản sao biến đổi ngẫu nhiên của mỗi ảnh (150 000 ảnh). Biến đổi chỉ áp dụng cho tập train; validation và test không bị đụng tới. Mỗi điều kiện chạy với ba seed.

Điều kiện	Số mẫu huấn luyện	Val (%)	Test (%)	Thời gian fit (s)
Không tăng cường	50 000	97.88 ± 0.42	98.02 ± 0.29	8
Tăng cường (×3 dữ liệu)	150 000	98.28 ± 0.09	98.30 ± 0.13	24

Bảng 7. MLP với cấu hình đã chọn, huấn luyện lại với 3 seed (42, 43, 44); trung bình ± độ lệch chuẩn mẫu. Ảnh tăng cường: xoay tối đa 10°, dịch tối đa 2 pixel, co giãn 0.9–1.1; 2 bản sao cho mỗi ảnh gốc, chỉ từ tập train.

Trung bình test tăng nhẹ khi có tăng cường, và độ lệch chuẩn giữa các seed nhỏ hơn. Nhưng với ba seed, mức tăng này cùng bậc với độ lệch chuẩn của điều kiện gốc, nên chúng tôi chỉ kết luận là *có xu hướng cải thiện*, chưa đủ bằng chứng để khẳng định. Thời gian huấn luyện tăng theo cỡ tập

dữ liệu. Mô hình dùng trong bản demo và các bảng chính vẫn là MLP huấn luyện trên dữ liệu gốc.

8.2 Bản demo web và quy trình triển khai

Mô hình được xuất ra một "bundle" gồm ONNX cho Naïve Bayes, softmax và MLP (đồ thị dựng tay bằng `onnx.helper`, bước chuẩn hóa $(x - \bar{x})/(s + \varepsilon)$ nằm *trong* đồ thị nên tiền xử lý huấn luyện và suy luận trùng nhau) và JSON cho các cây của AdaBoost (bộ đánh giá nhỏ viết bằng TypeScript). Mỗi tệp xuất được kiểm tra số học trên toàn bộ 10 000 ảnh test: Gaussian NB: sai lệch xác suất lớn nhất $1.8e-04$ trùng nhãn 100.00% trên 10 000 ảnh; Softmax regression: sai lệch xác suất lớn nhất $3.4e-06$ trùng nhãn 100.00% trên 10 000 ảnh; MLP: sai lệch xác suất lớn nhất $1.9e-06$ trùng nhãn 100.00% trên 10 000 ảnh; AdaBoost-SAMME: sai lệch xác suất lớn nhất $0.0e+00$ trùng nhãn 100.00% trên 1 000 ảnh. Bundle cùng model card được đẩy lên Hugging Face Hub (không đưa trọng số lên GitHub). Trang web (React + TypeScript + Vite + ONNX Runtime Web, hoàn toàn tĩnh) cho phép vẽ hoặc tải ảnh, xem ảnh 28×28 sau tiền xử lý, chọn một trong bốn mô hình, xem top-k xác suất và bảng kết quả/ma trận nhầm lẫn/đường cong; ảnh vẽ được cắt, thu về hộp 20×20 và căn trọng tâm về $(14, 14)$ đúng như quy trình tạo MNIST. Quy trình CI (GitHub Actions) chạy kiểm tra mã, kiểm thử đơn vị, một lượt chạy thử nhỏ của pipeline và build giao diện; không huấn luyện lại MNIST ở mỗi commit.

9 Kết luận

Nhóm đã tự cài đặt và so sánh bốn bộ phân lớp trên MNIST với cùng một giao thức không rò rỉ. MLP cho kết quả tốt nhất (98.24% trên test) với chi phí nhỏ; AdaBoost-SAMME gần sát (97.60%) nhưng đắt hơn nhiều về thời gian huấn luyện và dự đoán; softmax (92.52%) là lựa chọn nhanh và dễ đọc; Naïve Bayes (81.32%) là mốc đơn giản nhất và bộc lộ rõ cái giá của giả thiết độc lập. Kết quả của các bản tự cài đặt khớp scikit-learn ở mức cho phép của từng thuật toán, và bản xuất ONNX khớp bản Python.

Bài học thực nghiệm lớn nhất: phần lớn lợi ích đến từ *chọn cấu hình bằng validation một cách có kỷ luật* (ví dụ `var_smoothing` cho Naïve Bayes, độ sâu cây cho AdaBoost, hình dạng mạng cho MLP), và từ việc ghi lại trung thực cả những thí nghiệm không rõ ràng (tăng cường dữ liệu chỉ cải thiện nhẹ và chưa đủ để kết luận chắc chắn với ba seed).

Hướng tiếp theo. Chạy nhiều seed và nhiều phép chia cho mọi mô hình để có khoảng tin cậy thật; tìm kiếm tương tác siêu tham số (ví dụ tìm ngẫu nhiên hoặc Bayes) trong cùng ngân sách; thử một mạng tích chập như phần mở rộng tùy chọn; hiệu chỉnh xác suất của AdaBoost; và kiểm tra bản demo trên dữ liệu viết tay thật ngoài MNIST.

10 Đóng góp của nhóm và công bố sử dụng AI

10.1 Phân công và đóng góp

Bảng dưới đây chỉ ghi những gì có thể kiểm chứng từ lịch sử Git của kho mã (`git shortlog`, các pull request #1–#4 và tệp `WORKFLOW.md`). Tỉ lệ đóng góp **50%–50%** do hai thành viên tự thống nhất và xác nhận (không suy ra từ số commit).

Thành viên	MSSV	Công việc có bằng chứng trong kho mã	Tỉ lệ
Lâm Nhật Tiến (GitHub: Tien-Lam28)	24280029	23 commit (13/10–06/11/2026) và PR #1–#4: dựng kho, README/WORKFLOW, tải và tiền xử lý dữ liệu, lớp cha `BaseClassifier`, độ đo, `Evaluator`, K-fold, Gaussian NB, bản đầu của MLP (backprop viết tay, early stopping), EDA, notebook trực quan. Theo `WORKFLOW.md` được phân công NB và MLP.	50%
Trần Trinh Nhân (GitHub: TranTrinhNhan1)	24280088	Khởi tạo kho; theo `WORKFLOW.md` được phân công mô hình tuyến tính, ensemble và phần mở rộng. Giai đoạn hoàn thiện (softmax regression, AdaBoost-SAMME, tái cấu trúc MLP/độ đo/tiền xử lý, bộ kiểm thử, pipeline thực nghiệm, xuất mô hình, website, báo cáo, slide) được thực hiện cùng trợ lý AI (xem dưới) và commit dưới danh nghĩa này.	50%

Bảng 8. Đóng góp theo bằng chứng Git. Họ tên và MSSV do nhóm cung cấp; tỉ lệ đóng góp (50%/50%) do nhóm xác nhận.

10.2 Công bố sử dụng công cụ AI

Theo yêu cầu của đề bài, nhóm công bố: phần hoàn thiện đồ án sau commit `f9a602b` – gồm `SoftmaxRegression`, `AdaBoostSAMME` và cây quyết định có trọng số, các sửa đổi cho `MLP`, độ đo, tiền xử lý và `Evaluator`, toàn bộ bộ kiểm thử, pipeline thực nghiệm và tích hợp Weights & Biases, quy trình xuất ONNX/Hugging Face, website, notebook tái lập, mã nguồn LaTeX của báo cáo này và slide – được viết *với sự hỗ trợ của trợ lý lập trình AI Claude Code (Anthropic)*. Trợ lý đã chạy mã, kiểm thử và thí nghiệm trong môi trường của nhóm; mọi con số trong báo cáo được sinh từ các tệp kết quả do chương trình ghi ra. Các thành viên có trách nhiệm đọc, hiểu và giải thích được mã. Mức độ hỗ trợ của AI ở các giai đoạn trước đó (trước commit `f9a602b`) **[nhóm tự khai báo]**. Các tài liệu tham khảo được kiểm tra thủ công về tồn tại và thông tin thư mục.

Tài liệu tham khảo

[1] Lukas Biewald. Experiment tracking with Weights and Biases, 2020. Software available from https://www.wandb.com/.

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[3] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[4] Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification*. Wiley, 2 edition, 2001.

[5] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[7] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning

applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[8] Lutz Prechelt. Early stopping – but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, 1998.

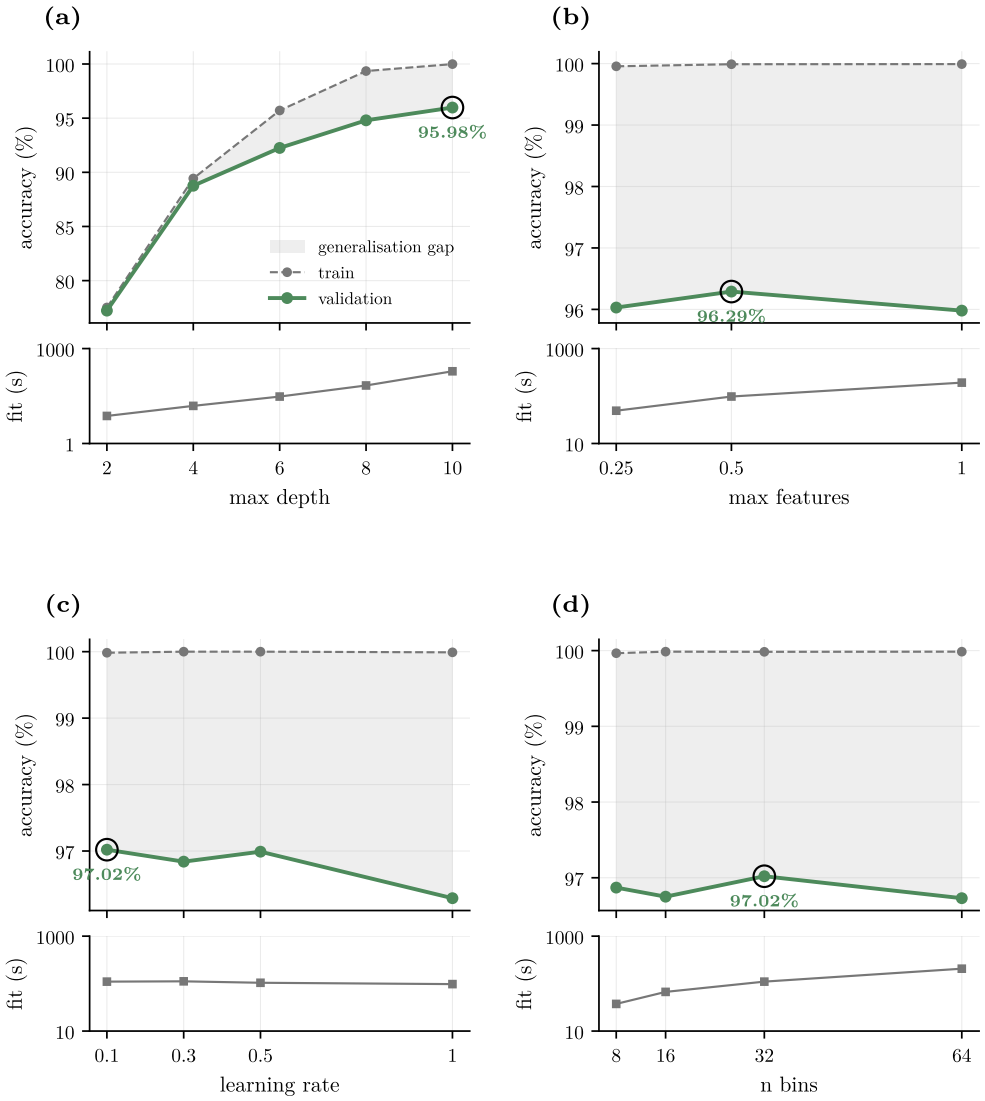
[9] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

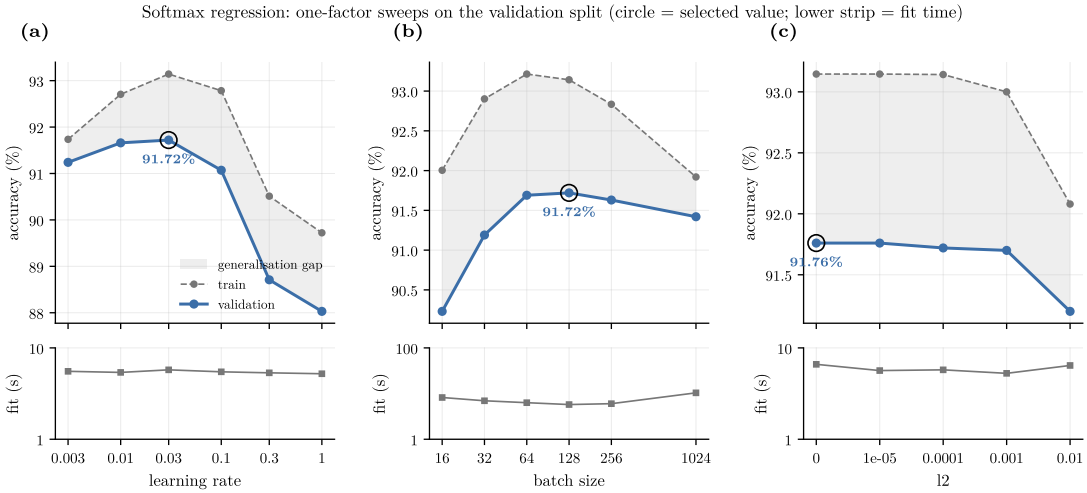
[11] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28 of *PMLR*, pages 1139–1147, 2013.

[12] Ji Zhu, Hui Zou, Saharon Rosset, and Trevor Hastie. Multi-class AdaBoost. *Statistics and Its Interface*, 2(3):349–360, 2009.

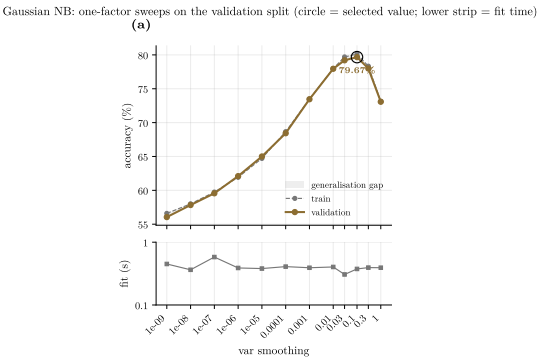
A Các lượt quét siêu tham số còn lại



Hình 8. AdaBoost-SAMME: quét từng siêu tham số trên validation (số vòng boosting chọn riêng từ đường cong, hình 5c).



Hình 9. Hồi quy softmax: quét từng siêu tham số trên validation.



Hình 10. Gaussian Naïve Bayes: quét `var_smoothing` trên validation.

B Tái lập kết quả

Môi trường dùng `uv` (Python $\geq$ 3.11); mã, kiểm thử và dòng lệnh nằm trong kho GitHub của đồ án.

```
uv sync --extra track --extra export
uv run python -m src prepare          # tải MNIST, chia train/val, chuẩn hóa
uv run python -m src tune --model all # quét siêu tham số, ghi results/tuning
uv run python -m src train --model all
uv run python -m src baselines --model all   # scikit-learn
uv run python -m src augment          # thí nghiệm tăng cường dữ liệu
uv run python -m src report           # bảng, hình, các đoạn LaTeX
uv run python -m src export           # ONNX + kiểm tra số học
uv run python -m src upload           # đẩy lên Hugging Face (cần token)
uv run pytest
```

Báo cáo biên dịch bằng `pdflatex` + `bibtex` trong thư mục `report/`; các bảng và số liệu nằm trong `results/tables` và `report/generated` do bước `report` sinh ra.